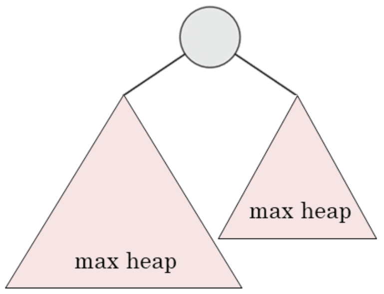
طراحی الگوریتم‌ها – مرتب‌سازی سریع

Introduction to Algorithm

مهدی جوانمردی

مرور جلسه قبل

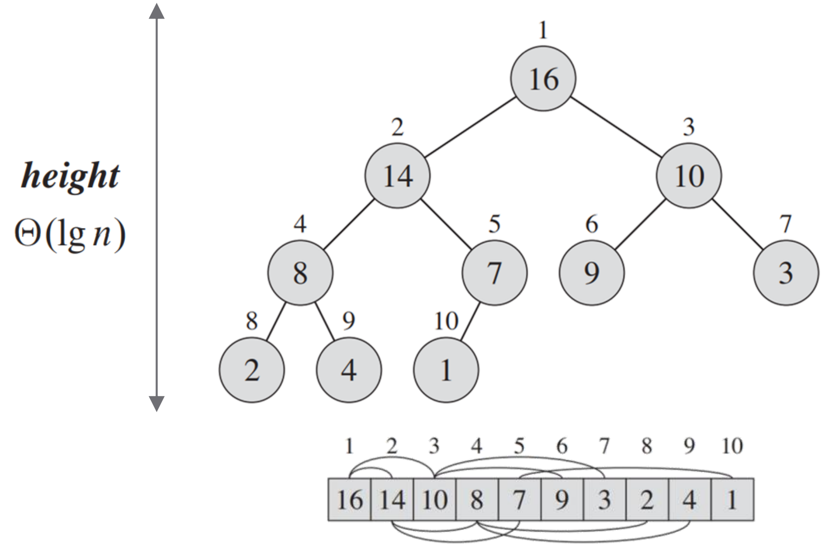
MAX-HEAPIFY



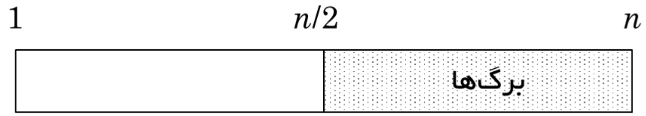
$\boxed{T(n) = O(\lg n)}$

```
MAX-HEAPIFY(A, i)
    l = LEFT(i)
    r = RIGHT(i)
    if l ≤ A.heap-size and A[l] > A[i]
        largest = l
    else largest = i
    if r ≤ A.heap-size and A[r] > A[largest]
        largest = r
    if largest ≠ i
        exchange A[i] with A[largest]
        MAX-HEAPIFY(A, largest)
```

هرم یا HEAP



BUILD-HEAP & HEAPSORT




```
BUILD-MAX-HEAP(A)
    A.heap-size = A.length
    for i = ⌊A.length/2⌋ downto 1
        MAX-HEAPIFY(A, i)
```

$O(n)$

```
HEAPSORT(A)
    BUILD-MAX-HEAP(A)                 O(n)
    for i = A.length downto 2
        exchange A[1] with A[i]
        A.heap-size = A.heap-size - 1
        MAX-HEAPIFY(A, 1)             O(lg n)
```

n times

$O(n \lg n)$

عملیات‌های روی HEAP

Maintain/Restore the max-heap property $O(\lg n)$ $O(h)$
MAX-HEAPIFY

Create a max-heap from an unordered array $O(n)$
BUILD-MAX-HEAP

Sort an array in place $O(n \lg n)$
HEAPSORT

Priority queues

فصل هفتم: مرتب‌سازی سریع | Quicksort

- معرفی Quicksort
- بازدهی Quicksort
- نسخه تصادفی Quicksort
- تحلیل Quicksort

II ***Sorting and Order Statistics***

مقایسه الگوریتم‌های مرتب‌سازی

only a constant number of elements of the input are ever stored outside the array.

• الگوریتم Insertion sort مرتبه زمانی worst-case running time $\Theta(n^2)$ expected running time $\Theta(n^2)$	sorts ***in place***	fast in-place sorting algorithm for small input
• الگوریتم Merge sort مرتبه زمانی running time $\Theta(n \lg n)$	~~sorts ***in place***~~ MERGE procedure	
• الگوریتم Heap sort مرتبه زمانی worst-case running time $O(n \lg n)$	sorts ***in place***	important data structure, called a heap priority queue
• الگوریتم Quick sort مرتبه زمانی worst-case running time $\Theta(n^2)$ expected running time $\Theta(n \lg n)$ outperforms heapsort in practice	sorts ***in place***	quicksort has tight code. popular algorithm for sorting large input arrays

مقایسه الگوریتم‌های مرتب‌سازی

we can beat this lower bound of $\Omega(n \lg n)$

if we can gather information about the sorted order of the input

- الگوریتم Counting sort

مرتبه زمانی

worst-case running time $\Theta(k + n)$

expected running time $\Theta(k + n)$

the input numbers are in the set $\{0, 1, \ldots, k\}$

- الگوریتم Radix sort

مرتبه زمانی

worst-case running time $\Theta(d(n + k))$

expected running time $\Theta(d(n + k))$

there are n integers to sort

integer has d digits

digit can take on up to k possible values

- الگوریتم Bucket sort

مرتبه زمانی

worst-case running time $\Theta(n^2)$

average-case running time $\Theta(n)$

requires knowledge of the probabilistic distribution of numbers in the input array

real numbers uniformly distributed in the half-open interval $[0, 1)$

مرتب‌سازی سریع یا Quicksort

quicksort has tight code.
popular algorithm for sorting large input arrays

sorts *in place*

worst-case running time $\Theta(n^2)$
expected running time $\Theta(n \lg n)$
outperforms heapsort in practice

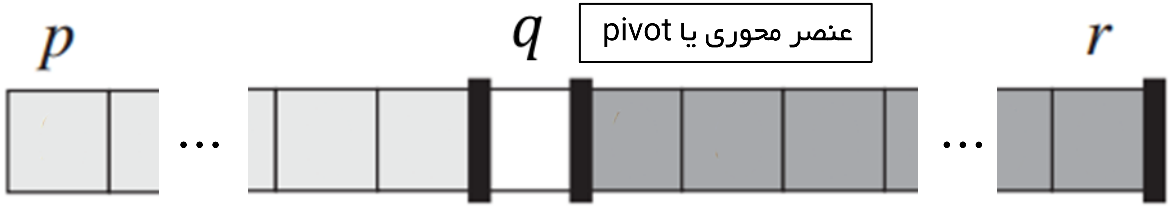
مرتبه زمانی

- Quicksort با وجود بدترین زمان اجرا $O(n^2)$ معمولا بهترین انتخاب مرتب‌سازی
- به دلیل expected خوب و ثابت‌های خیلی کوچک در تقریب O و درجا بودن عملیات
- به دلیل درجا بودن عملیات و جابجایی کم مناسب برای حافظه‌های مجازی Virtual Memory
- مشابه Mergesort از روش تقسیم و حل استفاده میکند

نحوه کار Quicksort

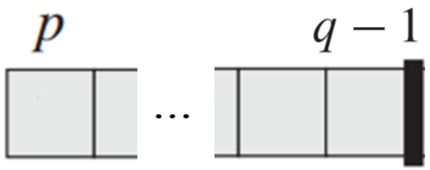
Partition (rearrange) the array $A[p \,.\,.\, r]$

تقسیم



$$A[p \,.\,.\, q-1] \;\le\; A[q] \;\le\; A[q+1 \,.\,.\, r]$$

حل

Quicksort(

)

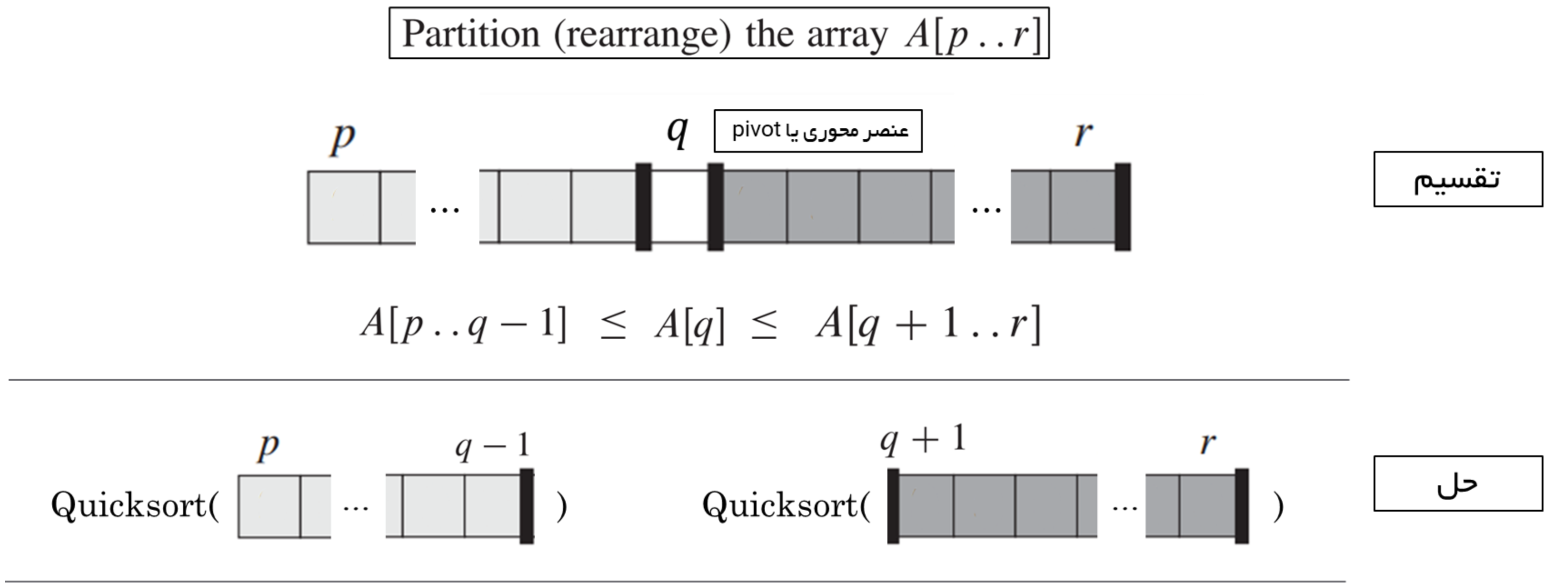
Quicksort(

)

ترکیب

$A[p \,.\,.\, r]$ is now sorted

شبه کد Quicksort بصورت بازگشتی

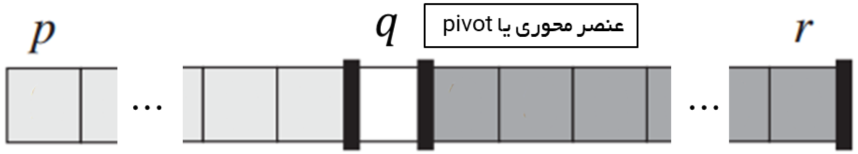



```
QUICKSORT(A, p, r)
if p < r
    q = PARTITION(A, p, r)
    QUICKSORT(A, p, q − 1)
    QUICKSORT(A, q + 1, r)
```

اولین فراخوانی تابع:

$\boxed{\text{QUICKSORT}(A, 1, A.\mathit{length}).}$

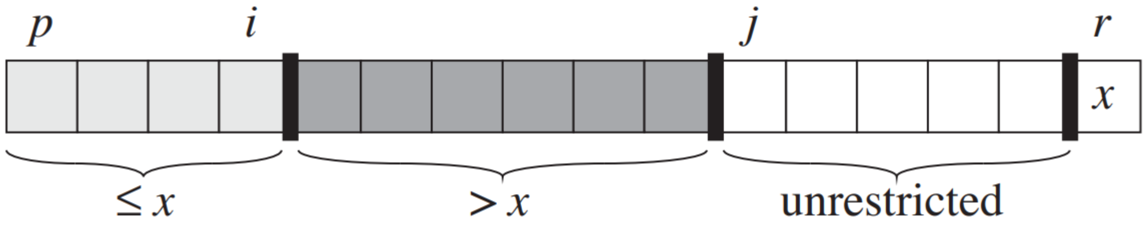
نحوه کار Partition



$$A[p \,..\, q-1] \;\le\; A[q] \;\le\; A[q+1 \,..\, r]$$

- انتخاب آخرین عنصر به عنوان pivot
- تغییر چیدمان عناصر آرایه با رعایت قوانین چهار بخش
- قرارداد pivot در جایگاه مناسب

```
PARTITION(A, p, r)
x = A[r]
i = p - 1
for j = p to r - 1
    if A[j] ≤ x
        i = i + 1
        exchange A[i] with A[j]
exchange A[i + 1] with A[r]
return i + 1
```

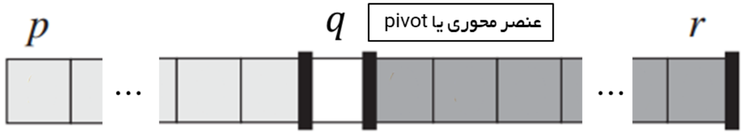


INTRODUCTION TO ALGORITHM | درس طراحی الگوریتم

دانشگاه صنعتی امیرکبیر (پلی تکنیک تهران)

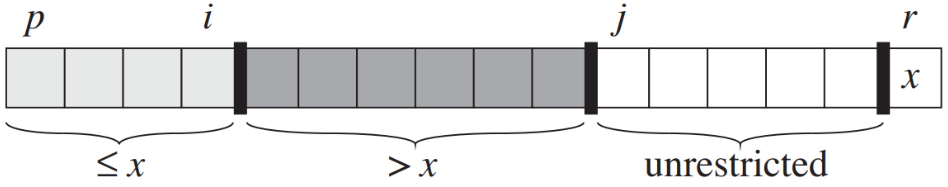
نمونه Partition

هدف نهایی

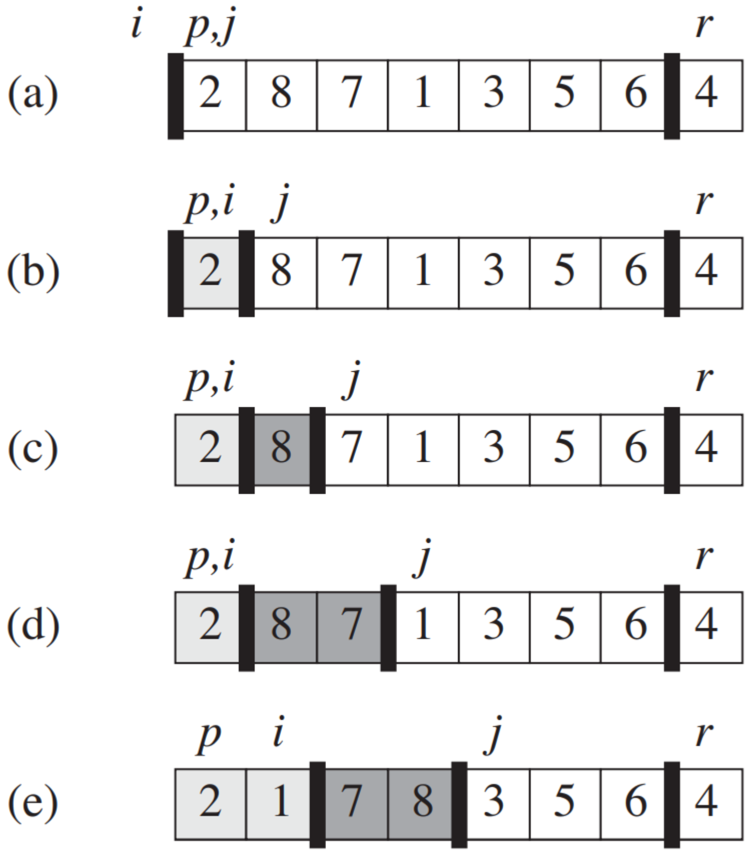


$A[p..q-1] \le A[q] \le A[q+1..r]$

شرایط میانی

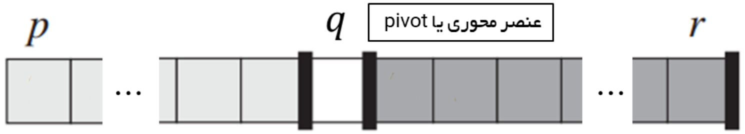



```
PARTITION(A, p, r)
x = A[r]
i = p - 1
for j = p to r - 1
    if A[j] ≤ x
        i = i + 1
        exchange A[i] with A[j]
exchange A[i + 1] with A[r]
return i + 1
```



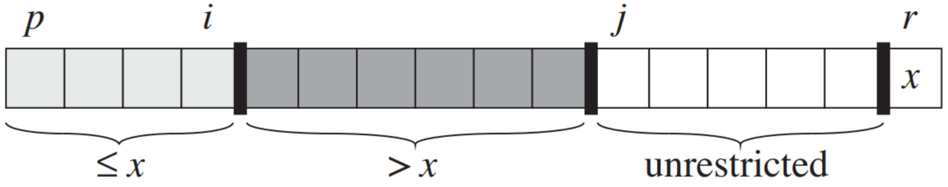
نمونه Partition

هدف نهایی

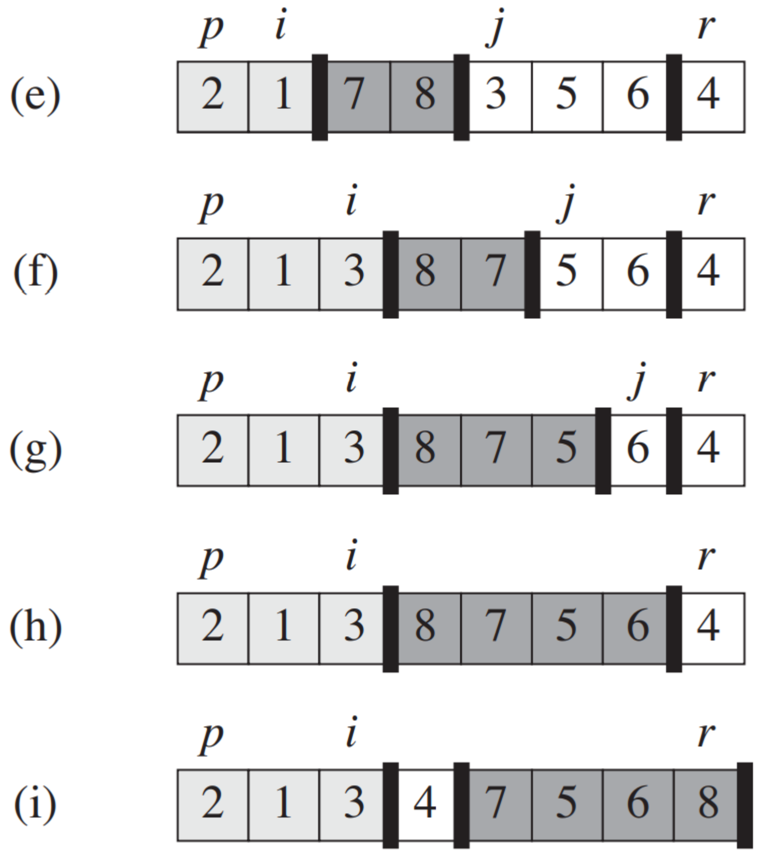


$$A[p\,..\,q-1] \le A[q] \le A[q+1\,..\,r]$$

شرایط میانی

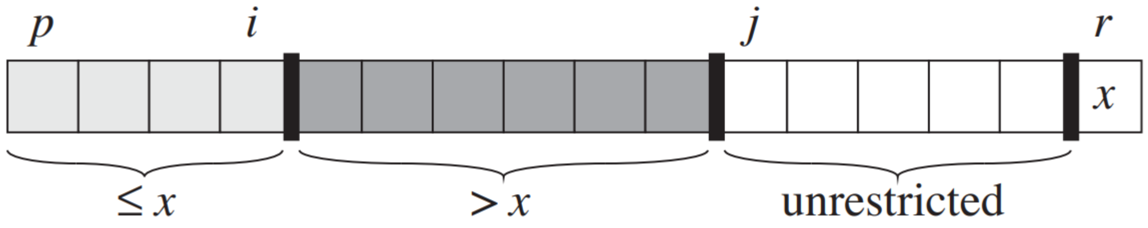



```
PARTITION(A, p, r)
x = A[r]
i = p - 1
for j = p to r - 1
    if A[j] ≤ x
        i = i + 1
        exchange A[i] with A[j]
exchange A[i + 1] with A[r]
return i + 1
```



اثبات شبه کد Partition

```
PARTITION(A, p, r)
1  x = A[r]
2  i = p − 1
3  for j = p to r − 1
4      if A[j] ≤ x
5          i = i + 1
6          exchange A[i] with A[j]
7  exchange A[i + 1] with A[r]
8  return i + 1
```



مستقل از حلقه

At the beginning of each iteration of the loop of lines 3–6, for any array index k,

1. If $p \le k \le i$, then $A[k] \le x$.
2. If $i + 1 \le k \le j - 1$, then $A[k] > x$.
3. If $k = r$, then $A[k] = x$.

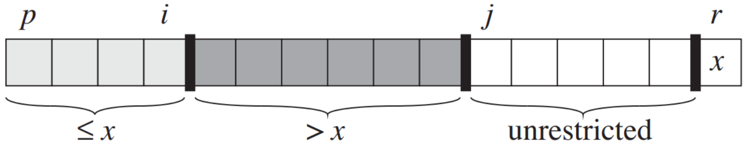
اثبات شبه کد Partition

```
PARTITION(A, p, r)
1  x = A[r]
2  i = p − 1
3  for j = p to r − 1
4      if A[j] ≤ x
5          i = i + 1
6          exchange A[i] with A[j]
7  exchange A[i + 1] with A[r]
8  return i + 1
```

مستقل از حلقه

At the beginning of each iteration of the loop of lines 3–6, for any array index k,

1. If $p \le k \le i$, then $A[k] \le x$.
2. If $i + 1 \le k \le j - 1$, then $A[k] > x$.
3. If $k = r$, then $A[k] = x$.



Initialization: $i = p - 1$ and $j = p$

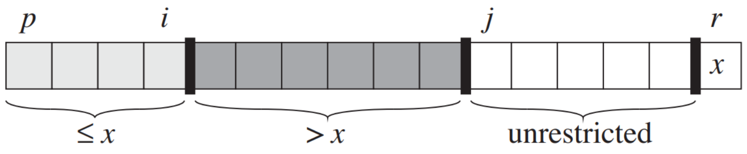
اثبات شبه کد Partition

```
PARTITION(A, p, r)
1  x = A[r]
2  i = p - 1
3  for j = p to r - 1
4      if A[j] ≤ x
5          i = i + 1
6          exchange A[i] with A[j]
7  exchange A[i + 1] with A[r]
8  return i + 1
```

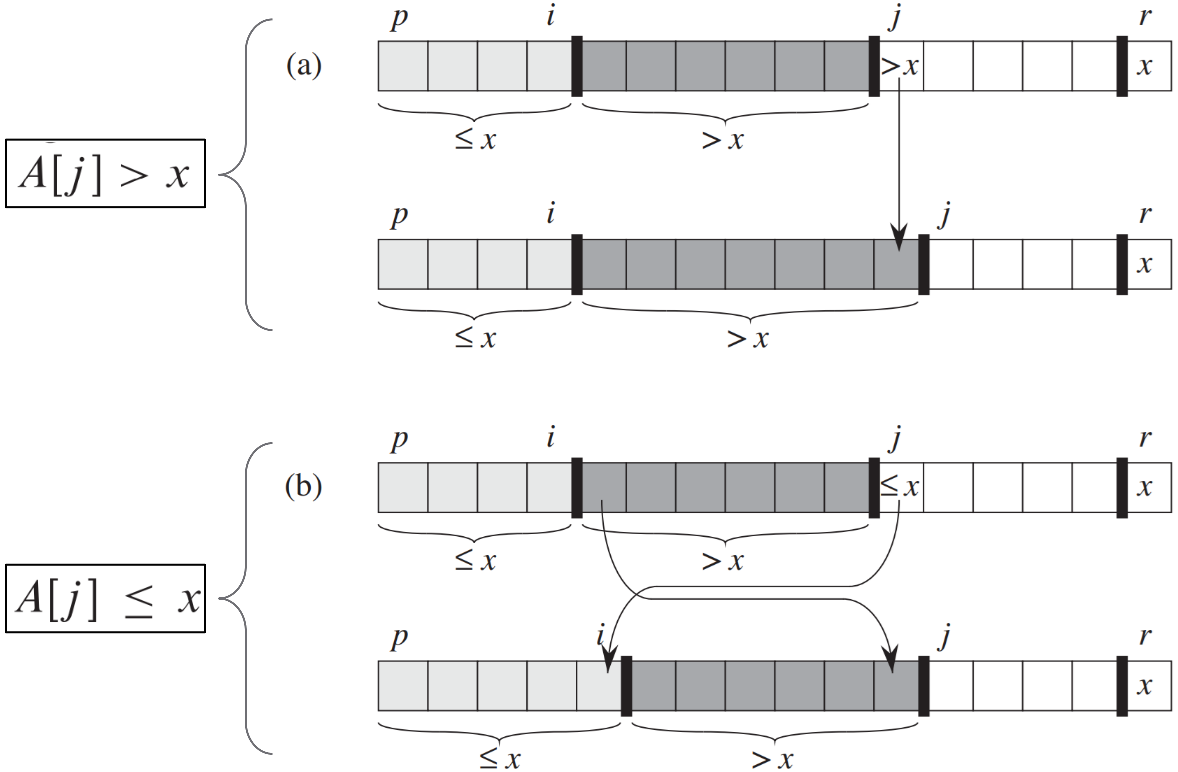
مستقل از حلقه

At the beginning of each iteration of the loop of lines 3–6, for any array index k,

1. If $p \le k \le i$, then $A[k] \le x$.
2. If $i + 1 \le k \le j - 1$, then $A[k] > x$.
3. If $k = r$, then $A[k] = x$.



Maintenance: $\boxed{j-1} \xrightarrow{} \boxed{j}$ (✓ above $j-1$, ? above j)



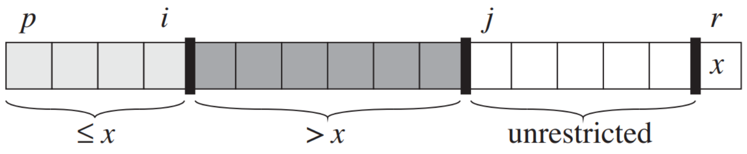
اثبات شبه کد Partition

```
PARTITION(A, p, r)
1  x = A[r]
2  i = p - 1
3  for j = p to r - 1
4      if A[j] ≤ x
5          i = i + 1
6          exchange A[i] with A[j]
7  exchange A[i + 1] with A[r]
8  return i + 1
```

مستقل از حلقه

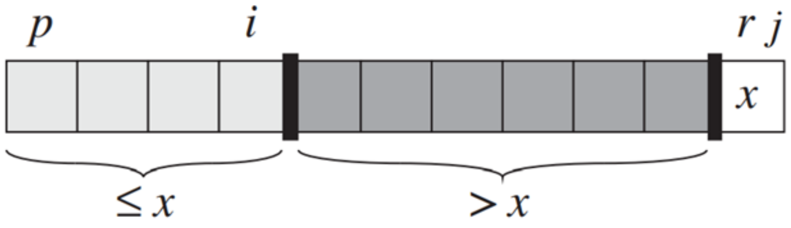
At the beginning of each iteration of the loop of lines 3–6, for any array index k,

1. If $p \le k \le i$, then $A[k] \le x$.
2. If $i + 1 \le k \le j - 1$, then $A[k] > x$.
3. If $k = r$, then $A[k] = x$.

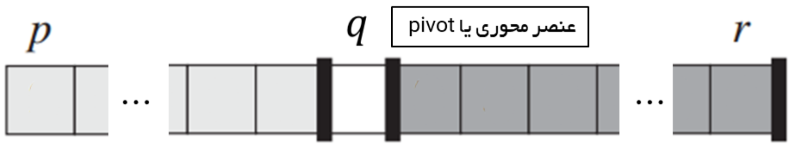


Termination: $j = r$

three sets:



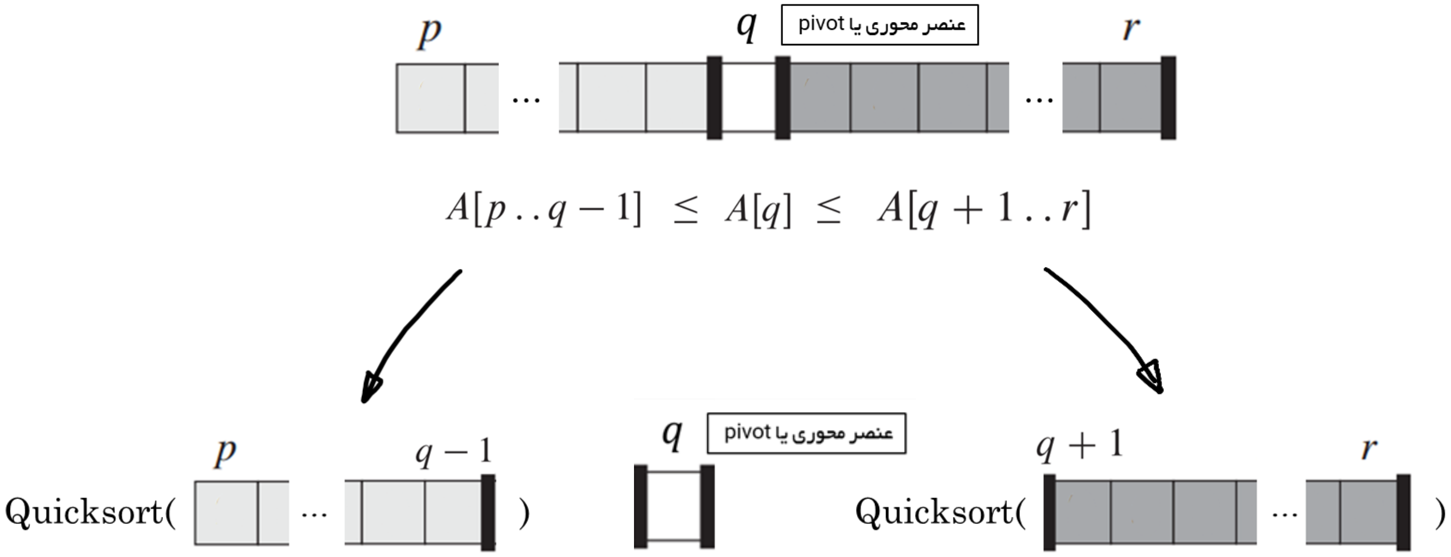
$\boxed{7 \quad \text{exchange } A[i+1] \text{ with } A[r]}$



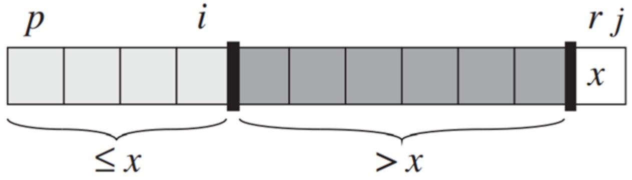
$$A[p \,..\, q-1] \;\le\; A[q] \;\le\; A[q+1 \,..\, r]$$

تحلیل زمان اجرای Quicksort

```
PARTITION(A, p, r)
x = A[r]
i = p - 1
for j = p to r - 1
    if A[j] ≤ x
        i = i + 1
        exchange A[i] with A[j]
exchange A[i + 1] with A[r]
return i + 1
```

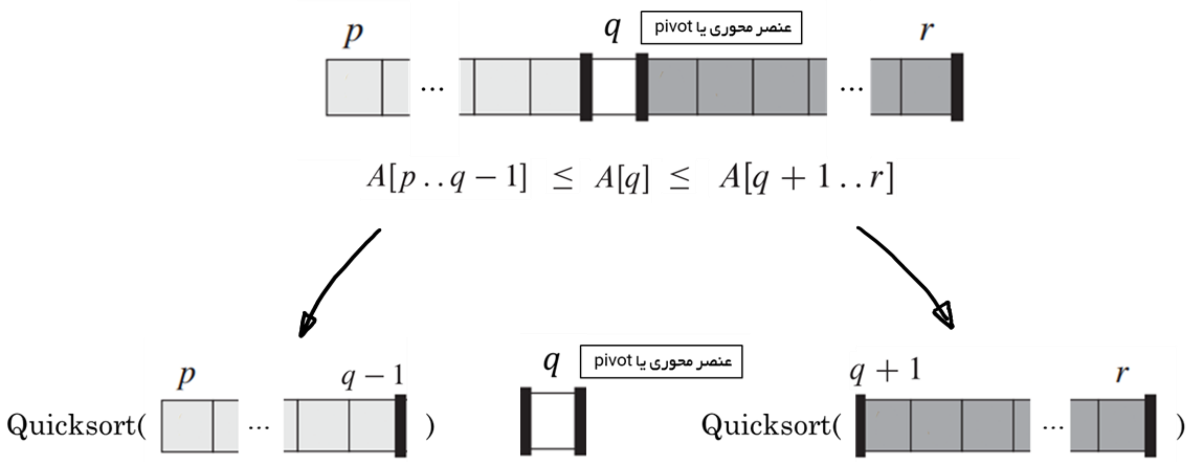


- پس از تقسیم، آرایه اولیه چقدر متقارن تقسیم شده است؟
- چه حالتی بهترست؟
- چه پارامتری تعیین کننده است؟



تحلیل شهودی بدتری زمان اجرای Quicksort

```
PARTITION(A, p, r)
x = A[r]
i = p − 1
for j = p to r − 1
    if A[j] ≤ x
        i = i + 1
        exchange A[i] with A[j]
exchange A[i + 1] with A[r]
return i + 1
```



دارای صفر عنصر

دارای $n-1$ عنصر

$$\begin{aligned} T(n) &= T(n-1) + T(0) + \Theta(n) \\ &= T(n-1) + \Theta(n)\ . \end{aligned}$$

$\longrightarrow$

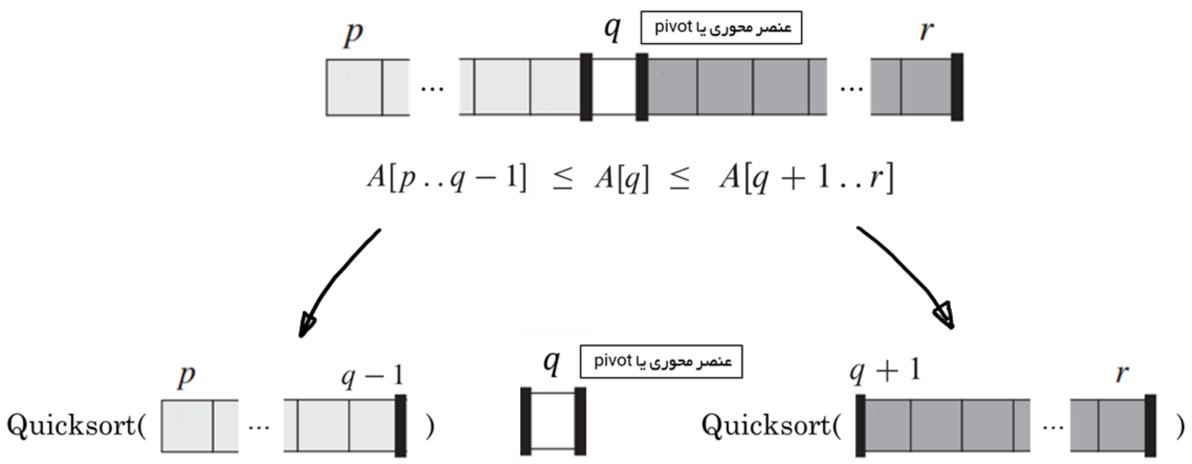
$$\begin{aligned} \sum_{k=1}^{n} k &= \frac{1}{2}n(n+1) \\ &= \Theta(n^2)\ . \end{aligned}$$

$\longrightarrow$ $\Theta(n^2)$

زمان اجرای quicksort برای آرایه مرتب؟ $\longleftarrow$ برای مرتب‌سازی درجی؟

تحلیل ریاضی بدترین زمان اجرای Quicksort

```
PARTITION(A, p, r)
x = A[r]
i = p − 1
for j = p to r − 1
    if A[j] ≤ x
        i = i + 1
        exchange A[i] with A[j]
exchange A[i + 1] with A[r]
return i + 1
```



دارای $n - q - 1$ عنصر

دارای q عنصر

$$T(n) = \max_{0 \le q \le n-1} (T(q) + T(n-q-1)) + \Theta(n) \longrightarrow \boxed{O(n^2)}$$

$$\boxed{T(n) \le cn^2}$$

$$\begin{aligned} T(n) &\le \max_{0 \le q \le n-1} (cq^2 + c(n-q-1)^2) + \Theta(n) \\ &= c \cdot \max_{0 \le q \le n-1} (q^2 + (n-q-1)^2) + \Theta(n) . \end{aligned}$$

second derivative of the expression with respect to q is positive

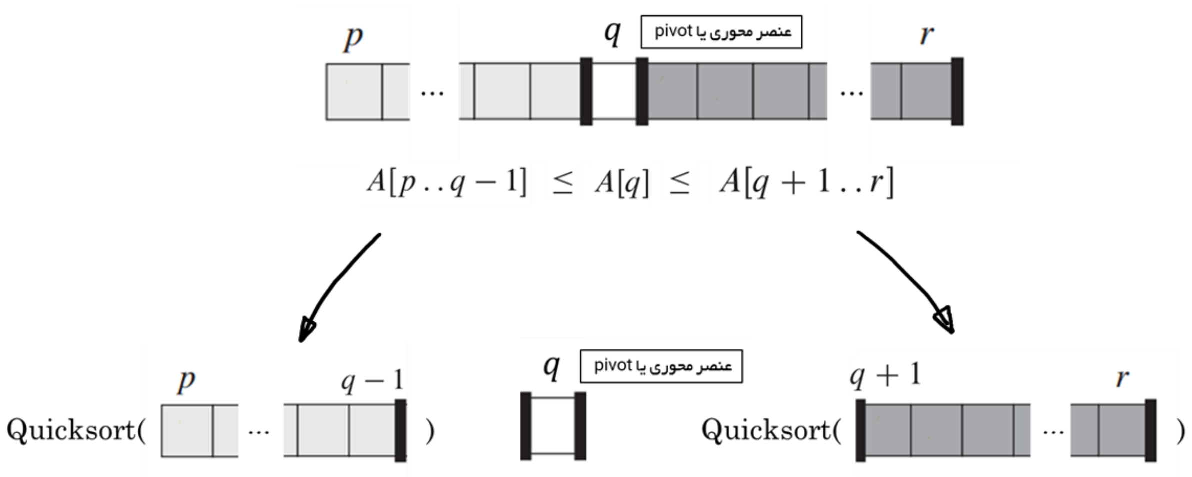
$$c \cdot \max_{0 \le q \le n-1} (q^2 + (n-q-1)^2) \le c \cdot (n-1)^2$$

$$\begin{aligned} T(n) &\le cn^2 - c(2n-1) + \Theta(n) \\ &\le cn^2 , \end{aligned}$$

$$\boxed{\text{if } c(2n-1) \ge \Theta(n) .}$$

تحلیل بهترین زمان اجرای Quicksort

```
PARTITION(A, p, r)
x = A[r]
i = p − 1
for j = p to r − 1
    if A[j] ≤ x
        i = i + 1
        exchange A[i] with A[j]
exchange A[i + 1] with A[r]
return i + 1
```



دارای $\lfloor n/2 \rfloor$ عنصر

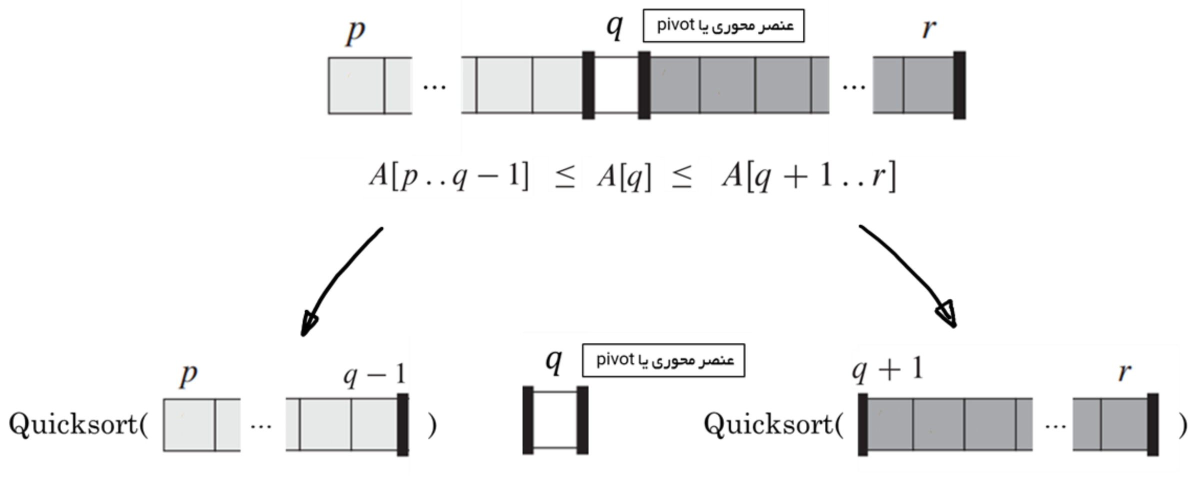
دارای $\lceil n/2 \rceil - 1$ عنصر

$$T(n) = 2T(n/2) + \Theta(n) \,, \longrightarrow \boxed{\text{حالت دوم قضیه اصلی}} \longrightarrow T(n) = \Theta(n \lg n)$$

تحلیل متوسط زمان اجرای Quicksort

متوسط زمان اجرای Quicksort به بهترین زمان اجرای آن بسیار نزدیک است

```
PARTITION(A, p, r)
x = A[r]
i = p − 1
for j = p to r − 1
    if A[j] ≤ x
        i = i + 1
        exchange A[i] with A[j]
exchange A[i + 1] with A[r]
return i + 1
```



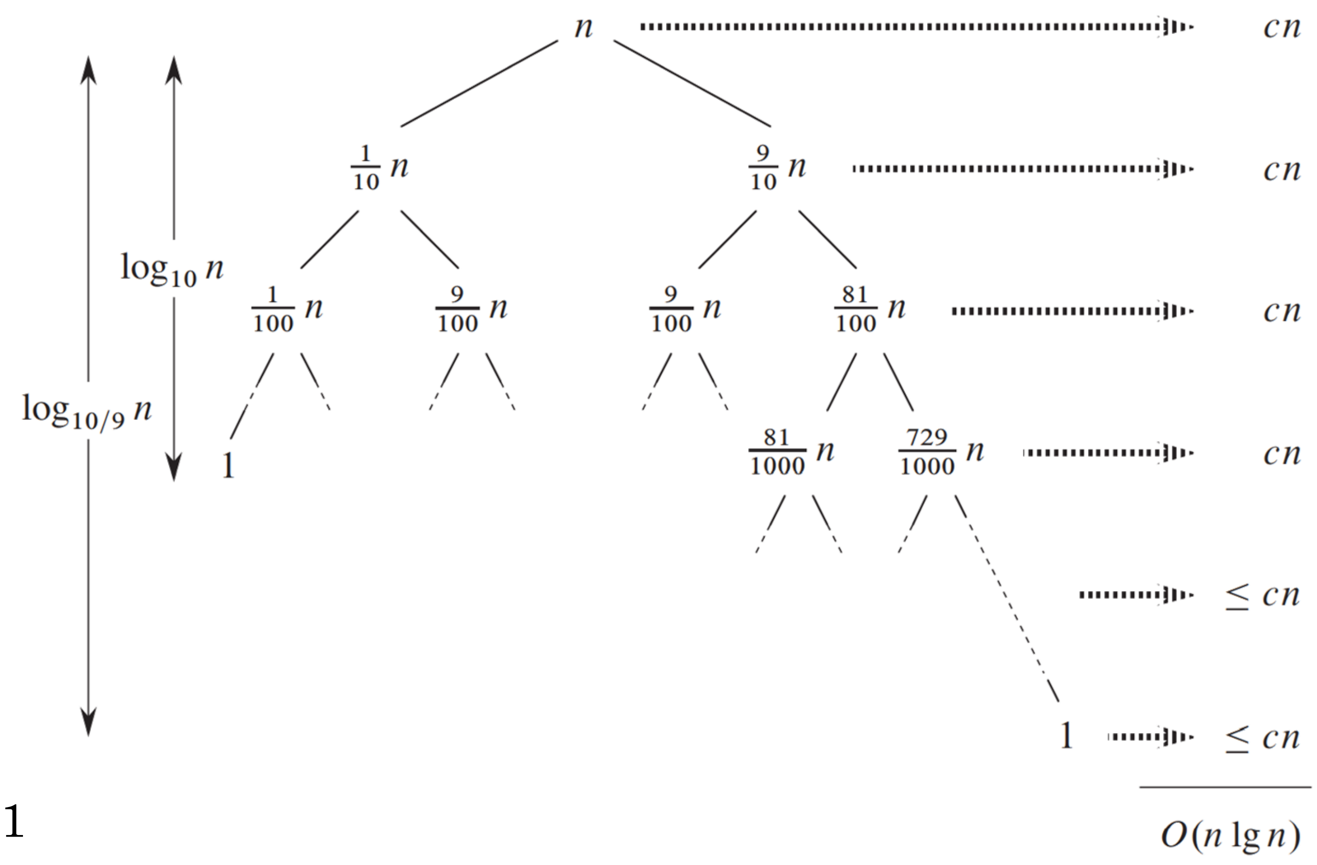
دارای $n/10$ عنصر

دارای $9n/10$ عنصر

$$T(n) = T(9n/10) + T(n/10) + cn$$ → حدس با درخت بازگشتی

تحلیل متوسط زمان اجرای Quicksort

$\boxed{T(n) = T(9n/10) + T(n/10) + cn}$ → حدس با درخت بازگشتی



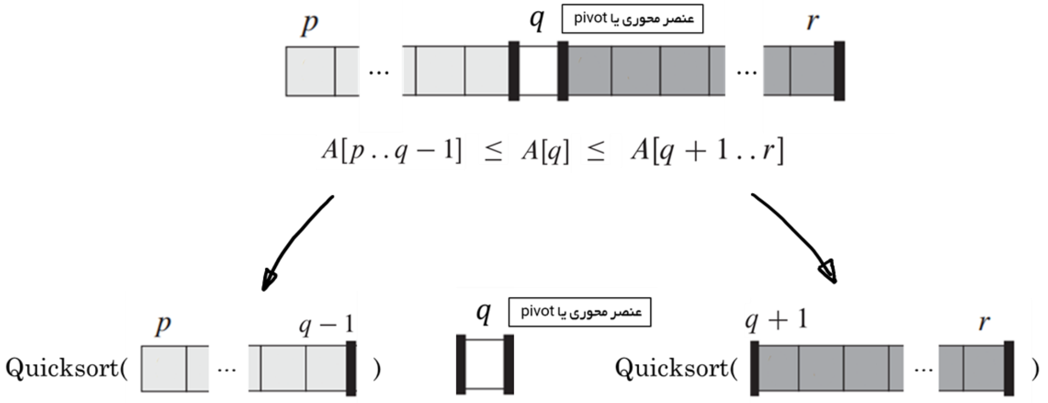
برای تقسیم ۹۹ به ۱ چطور؟؟

↓

برای تقسیم ثابت همواره $T(n) = \Theta(n \lg n)$

تحلیل شهودی متوسط زمان اجرای Quicksort


```
PARTITION(A, p, r)
x = A[r]
i = p - 1
for j = p to r - 1
    if A[j] ≤ x
        i = i + 1
        exchange A[i] with A[j]
exchange A[i + 1] with A[r]
return i + 1
```



برای محاسبه دقیق زمان متوسط اجرای Quicksort نیاز به فرضیاتی نسبت به چیدمان ورودی داریم

صرفا چیدمان اهمیت دارد و نه خود اعداد ورودی

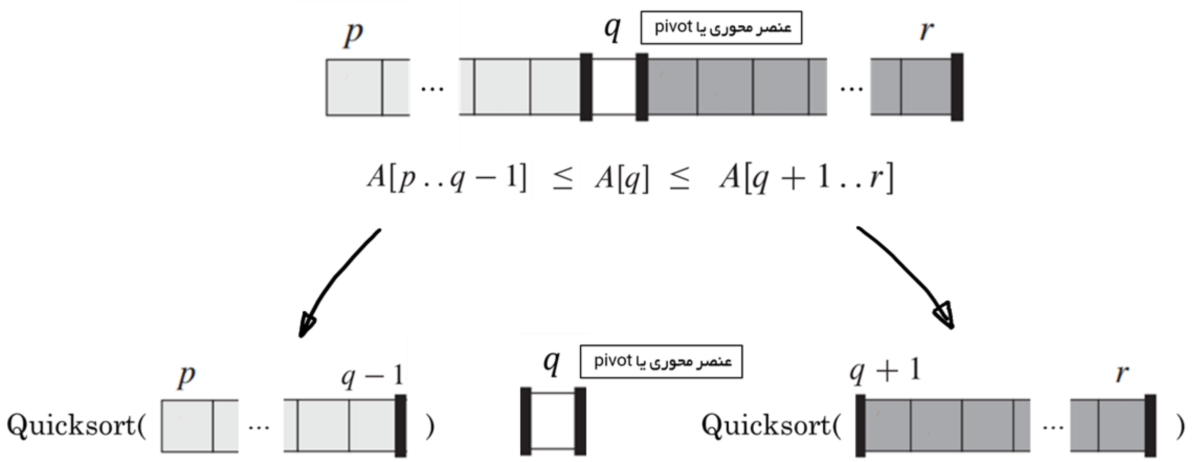
↓

فرض: تمام جایگشت‌های اعداد ورودی با احتمال یکسان رخ میدهند

$\boxed{\text{Exercise 7.2-6}}$ about 80 percent of the time PARTITION produces a split that is more balanced than 9 to 1

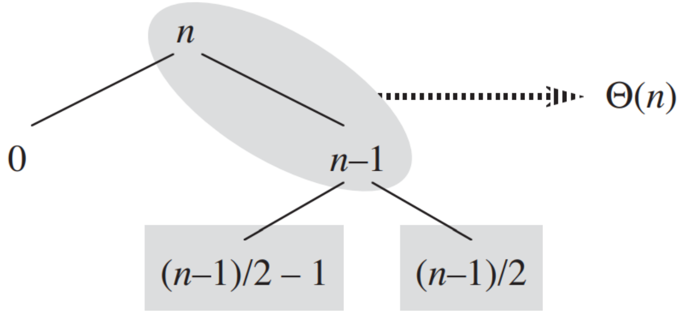
تحلیل شهودی متوسط زمان اجرای Quicksort

```
PARTITION(A, p, r)
x = A[r]
i = p − 1
for j = p to r − 1
    if A[j] ≤ x
        i = i + 1
        exchange A[i] with A[j]
exchange A[i + 1] with A[r]
return i + 1
```

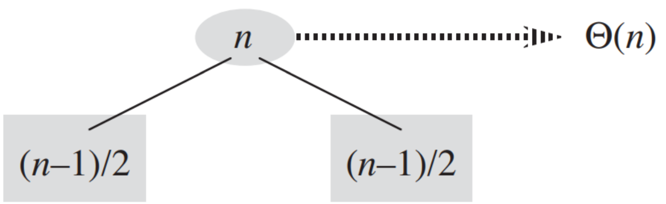


بار اول: دارای صفر عنصر — دارای $n-1$ عنصر

بار دوم: دارای $\lfloor n/2 \rfloor$ عنصر — دارای $\lceil n/2 \rceil - 1$ عنصر

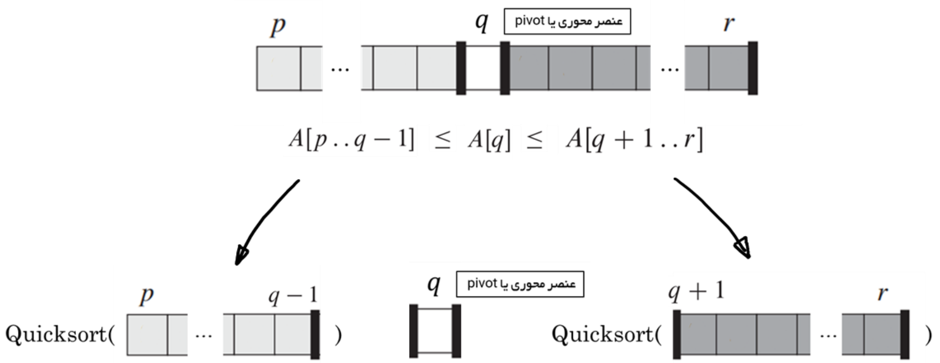


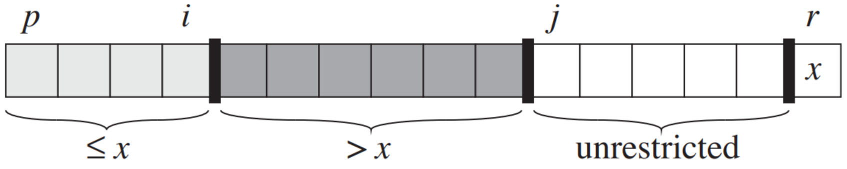
(a)



(b)

مدل تصادفی Quicksort





~~فرض: تمام جایگشت‌های اعداد ورودی با احتمال یکسان رخ میدهند~~

random sampling

Pivot selection: $A[r] \longrightarrow$ Randomly choose from $A[p\,..\,r]$

RANDOMIZED-PARTITION(A, p, r)

```
i = RANDOM(p, r)
exchange A[r] with A[i]
return PARTITION(A, p, r)
```

RANDOMIZED-QUICKSORT(A, p, r)

```
if p < r
    q = RANDOMIZED-PARTITION(A, p, r)
    RANDOMIZED-QUICKSORT(A, p, q − 1)
    RANDOMIZED-QUICKSORT(A, q + 1, r)
```

زمان اجرای متوسط Quicksort تصادفی

```
QUICKSORT(A, p, r)
if p < r
    q = PARTITION(A, p, r)
    QUICKSORT(A, p, q − 1)
    QUICKSORT(A, q + 1, r)
```

```
PARTITION(A, p, r)
x = A[r]
i = p − 1
for j = p to r − 1
    if A[j] ≤ x
        i = i + 1
        exchange A[i] with A[j]
exchange A[i + 1] with A[r]
return i + 1
```

- The dominant cost of the algorithm is partitioning.
- PARTITION removes the pivot element from future consideration each time.
- Thus, PARTITION is called at most n times.
- QUICKSORT recurses on the partitions.
- The amount of work that each call to PARTITION does is a constant plus the number of comparisons that are performed in its **for** loop.
- Let $X =$ the total number of comparisons performed in all calls to PARTITION.
- Therefore, the total work done over the entire execution is $O(n + X)$.

زمان اجرای متوسط Quicksort تصادفی

```
QUICKSORT(A, p, r)
if p < r
    q = PARTITION(A, p, r)
    QUICKSORT(A, p, q − 1)
    QUICKSORT(A, q + 1, r)
```

```
PARTITION(A, p, r)
x = A[r]
i = p − 1
for j = p to r − 1
    if A[j] ≤ x
        i = i + 1
        exchange A[i] with A[j]
exchange A[i + 1] with A[r]
return i + 1
```

We will now compute a bound on the overall number of comparisons.

For ease of analysis:

- Rename the elements of A as $z_1, z_2, \ldots, z_n$, with z_i being the ith smallest element.
- Define the set $Z_{ij} = \{z_i, z_{i+1}, \ldots, z_j\}$ to be the set of elements between z_i and z_j, inclusive.

Each pair of elements is compared at most once, because elements are compared only to the pivot element, and then the pivot element is never in any later call to PARTITION.

زمان اجرای متوسط Quicksort

```
QUICKSORT(A, p, r)
if p < r
    q = PARTITION(A, p, r)
    QUICKSORT(A, p, q - 1)
    QUICKSORT(A, q + 1, r)
```

```
PARTITION(A, p, r)
x = A[r]
i = p - 1
for j = p to r - 1
    if A[j] ≤ x
        i = i + 1
        exchange A[i] with A[j]
exchange A[i + 1] with A[r]
return i + 1
```

$$\mathrm{I}\{A\} = \begin{cases} 1 & \text{if } A \text{ occurs ,} \\ 0 & \text{if } A \text{ does not occur .} \end{cases}$$

Let $X_{ij} = \mathrm{I}\{z_i \text{ is compared to } z_j\}$.
(Considering whether z_i is compared to z_j at any time during the entire quicksort algorithm, not just during one call of PARTITION.)

Since each pair is compared at most once, the total number of comparisons performed by the algorithm is

$$X = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij} \ .$$

زمان اجرای متوسط Quicksort تصادفی

Since each pair is compared at most once, the total number of comparisons performed by the algorithm is

$$X = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij} \ .$$

Take expectations of both sides, use Lemma 5.1 and linearity of expectation:

$$\begin{aligned} \mathrm{E}[X] &= \mathrm{E}\left[\sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}\right] \\ &= \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \mathrm{E}[X_{ij}] \\ &= \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \Pr\{z_i \text{ is compared to } z_j\} \ . \end{aligned}$$

Now all we have to do is find the probability that two elements are compared.

زمان اجرای متوسط Quicksort

- Think about when two elements are *not* compared.
  - For example, numbers in separate partitions will not be compared.
  - In the previous example, $\langle 8, 1, 6, 4, 0, 3, 9, 5 \rangle$ and the pivot is 5, so that none of the set $\{1, 4, 0, 3\}$ will ever be compared to any of the set $\{8, 6, 9\}$.
- Once a pivot x is chosen such that $z_i < x < z_j$, then z_i and z_j will never be compared at any later time.
- If either z_i or z_j is chosen before any other element of Z_{ij}, then it will be compared to all the elements of Z_{ij}, except itself.
- The probability that z_i is compared to z_j is the probability that either z_i or z_j is the first element chosen.
- There are $j - i + 1$ elements, and pivots are chosen randomly and independently. Thus, the probability that any particular one of them is the first one chosen is $1/(j - i + 1)$.

دانشگاه صنعتی امیرکبیر
(پلی تکنیک تهران)

زمان اجرای متوسط Quicksort تصادفی

Therefore,

$$
\begin{aligned}
\Pr\{z_i \text{ is compared to } z_j\} &= \Pr\{z_i \text{ or } z_j \text{ is the first pivot chosen from } Z_{ij}\} \\
&= \Pr\{z_i \text{ is the first pivot chosen from } Z_{ij}\} \\
&\qquad + \Pr\{z_j \text{ is the first pivot chosen from } Z_{ij}\} \\
&= \frac{1}{j-i+1} + \frac{1}{j-i+1} \\
&= \frac{2}{j-i+1} .
\end{aligned}
$$

زمان اجرای متوسط Quicksort تصادفی

Substituting into the equation for $\mathrm{E}[X]$:

$$\mathrm{E}[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{2}{j-i+1} .$$

Evaluate by using a change in variables ($k = j - i$) and the bound on the harmonic series in equation (A.7):

$$\begin{aligned}
\mathrm{E}[X] &= \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{2}{j-i+1} \\
&= \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1} \\
&< \sum_{i=1}^{n-1} \sum_{k=1}^{n} \frac{2}{k} \\
&= \sum_{i=1}^{n-1} O(\lg n) \\
&= O(n \lg n) .
\end{aligned}$$